

hexens x ROYCO

Security Review Report for Royco

April 2026



Table of Contents

1. About Hexens

2. Executive summary

3. Security Review Details

- Security Review Lead
- Scope
- Changelog

4. Severity Structure

- Severity characteristics
- Issue symbolic codes

5. Findings Summary

6. Weaknesses

- Impossible to seize shares from requested withdrawn assets
- Users Can Cancel to Retain Yield or Execute for Immediate Withdrawal
- Maple redemptions are checked against RoycoEntryPoint instead of the original owner
- SoulBound tranche kernels are not compatible with RoycoEntryPoint async flows
- No slippage control on redemption flow
- Missing burn and burnFrom in deploy scripts
- Gas: Skip blacklist checks on transfers if not enabled
- Gas: Pre sync accounting should use cached variables
- Gas: Call internal functions in batch functions
- Gas: Remove redundant calculations

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report reviews an update to Royco, a perpetual risk-tranching protocol. The update focuses on a significant simplification of tranches and kernels, along with the introduction of an entry point contract that facilitates permissionless and asynchronous depositing and withdrawing.

Our security assessment was a full review of the new code, spanning a total of 1 week.

During our review, we did not identify any major severity vulnerabilities.

We did identify several minor issues and optimizations.

All reported issues were fixed or acknowledged by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- Review Led by

Kasper Zwijsen, Head of Audits

- Scope

The analyzed resources are located on:

-  ▪ PR 62: <https://github.com/roycoprotocol/royco-dawn/compare/f827c4786a721bd58ea690d3600ba7e47bbbed7ea...7b1d956eb001ab3dbcff88c2c101c516e521bbf9>
-  ▪ PR 63: <https://github.com/roycoprotocol/royco-dawn/compare/7b1d956eb001ab3dbcff88c2c101c516e521bbf9...35b2fc489950fcca249410d3b30aa6d12c717063>
-  ▪ PR 64: <https://github.com/roycoprotocol/royco-dawn/compare/7b1d956eb001ab3dbcff88c2c101c516e521bbf9...84946faf6ce33d908c68d7bcd35b7ab9f4fbce6b>

The issues described in this report were fixed in the following commit:

-  <https://github.com/roycoprotocol/royco-dawn/tree/373a9fbd083fd8baa30eaa1b5aba695ba9a84c77>

- Changelog

 3 April 2026	Audit start
 13 April 2026	Initial report
 20 April 2026	Revision received
 21 April 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity

Number of findings

Critical	0
High	0
Medium	0
Low	5
Informational	5

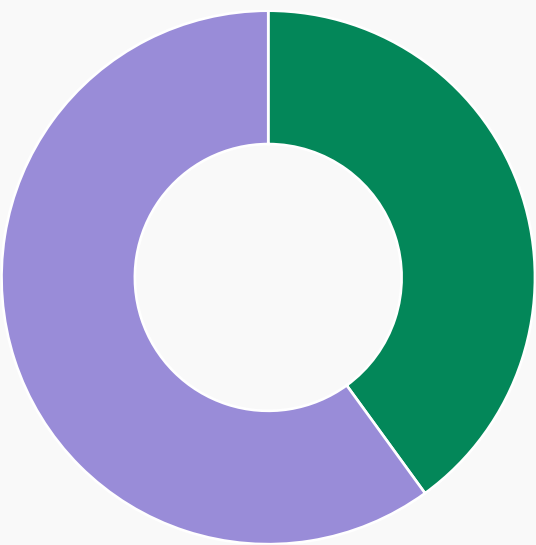
Total:

10



Low

Informational



Fixed

Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

ROYCO4-1 | Impossible to seize shares from requested withdrawn assets

Acknowledged

Severity:

Low

Probability:

Unlikely

Impact:

Low

Path:

src/periphery/RoycoEntryPoint.sol:requestRedemption#L221-L261

Description:

Whenever a user requests a redemption, the function **requestRedemption** transfers their tranche shares to **RoycoEntryPoint**. Because of this, seizing shares from users would currently become impossible.

```
function requestRedemption(
    address _tranche,
    uint256 _shares,
    address _receiver,
    uint64 _executorBonusWAD
)
    external
    override(IRoycoEntryPoint)
    whenNotPaused
    restricted
    returns (uint256 requestNonce, uint32 executableAtTimestamp)
{
    // Validate the redemption request
    require(_shares != 0, ZERO_AMOUNT());
    require(_tranche != address(0) && _receiver != address(0), NULL_ADDRESS());
    require(_executorBonusWAD <= WAD || _executorBonusWAD == type(uint64).max,
INVALID_EXECUTOR_BONUS());

    // Ensure that the tranche is enabled on this entry point
    RoycoEntryPointState storage $ = _getRoycoEntryPointStorage();
    EnrichedTrancheConfig memory config = $.trancheToConfig[_tranche];
```

```

require(config.baseConfig.enabled, TRANCHE_NOT_ENABLED());

// Register the user's redemption request with a fresh nonce
RedemptionRequest storage request = $.userToNonceToRedemptionRequest[msg.sender][requestNonce =
++$.lastRequestNonce];
request.shares = _shares;
// If the redeeming LP receives the yield accrued, set to MAX_NAV_UNITS so that navAtExecutionTime is
never greater, effectively disabling yield forfeiture
request.navAtRequestTime =
(config.baseConfig.yieldRecipient != AccruedYieldRecipient.REDEEMING_LP ?
IRoycoVaultTranche(_tranche).convertToAssets(_shares).nav : MAX_NAV_UNITS);
request.baseRequest = BaseRequest({
    tranche: _tranche,
    receiver: _receiver,
    executableAtTimestamp: (executableAtTimestamp = uint32(block.timestamp +
config.baseConfig.redemptionDelaySeconds)),
    executorBonusWAD: _executorBonusWAD
});

// Transfer the requested amount of tranche shares into the entry point to queue the redemption
IERC20(_tranche).safeTransferFrom(msg.sender, address(this), _shares);

// Emit the redemption request event
emit RedemptionRequested(msg.sender, requestNonce, _tranche, _shares, executableAtTimestamp,
_executorBonusWAD);
}

```

Remediation:

Consider implementing a way to seize the redemption request.

ROYCO4-13 | Users Can Cancel to Retain Yield or Execute for Immediate Withdrawal

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

```
-/src/periphery/RoycoEntryPoint.sol
```

Description:

When a user cancels a redemption request, they receive back the same number of tranche shares they originally locked. However, those shares may have increased in value due to yield accrued during the request's waiting period.

The yield forfeiture logic in `_redeemWithYieldRouting()` is only applied during `executeRedemption()` and is not triggered on cancellation. As a result, any accrued yield is not forfeited.

The root cause of the issue is that the user does not have to wait for the withdrawal delay, since they are already in the request state, and they can cancel to still get the upside (yield).

Impact:

A JT could deposit, and instantly call request redeem, and watch the position closely,

- If yield accrues, they cancel the request and recover shares that have appreciated in value and request withdrawal again.
- If losses occur, they proceed with a instant execution and redeem under the original request

For a kernel such as infinifi, where applied losses are only called a few times per day, a loss can be frontrun by a instant withdrawal, and the user would not suffer from this loss.

Remediation:

We recommend to forfeit the yield earned from the time the request was made until the cancel was called, this can be done in a similar way as in the redeem route to ensure cancel cannot be used to game the system.

Proof of Concept:

Add the test to the existing file `test/periphery/entrypoint/RoycoEntryPointMockTest.t.sol`

```
function test_cancelRedemption_success() public {

    uint256 depositAmount = 1000e18;

    // userA: deposits and requests redemption
    _depositToTranche(userA, address(stTranche), depositAmount);
    uint256 honestShares = stTranche.balanceOf(userA);

    vm.startPrank(userA);
    stTranche.approve(address(entryPoint), honestShares);
    (uint256 honestNonce,) = entryPoint.requestRedemption(address(stTranche), honestShares, userA, 0);
    vm.stopPrank();

    // userB deposits and requests redemption at the same time
    _depositToTranche(userB, address(stTranche), depositAmount);
    uint256 attackerShares = stTranche.balanceOf(userB);

    vm.startPrank(userB);
    stTranche.approve(address(entryPoint), attackerShares);
    (uint256 attackerNonce,) = entryPoint.requestRedemption(address(stTranche), attackerShares, userB, 0);
    vm.stopPrank();

    // 10% yield accrues during the delay period
    asset.mint(address(stTranche), (2 * depositAmount) / 10);
    stTranche.simulateYield(0.1e18);

    // forward past the redemption delay
    vm.warp(block.timestamp + REDEMPTION_DELAY + 1);

    // user A executes: yield is forfeited as intended
    vm.prank(userA);
    AssetClaims memory honestClaims = entryPoint.executeRedemption(userA, honestNonce,
type(uint256).max);
    uint256 honestReceived = toUint256(honestClaims.nav);

    // userB cancels instead of executing (avoids forfeiture)
    vm.startPrank(userB);
    entryPoint.cancelRedemptionRequest(attackerNonce, userB);

    // userB re-requests immediately, navAtRequestTime is now reset to the
```

```

// current higher NAV, so any prior yield is baked into the new baseline.
uint256 reRequestShares = stTranche.balanceOf(userB);
stTranche.approve(address(entryPoint), reRequestShares);
(uint256 newNonce,) = entryPoint.requestRedemption(address(stTranche), reRequestShares, userB, 0);
vm.stopPrank();

// Warp past the second delay period (no additional yield this time)
vm.warp(block.timestamp + REDEMPTION_DELAY + 1);

// userB executes during a flat period → zero yield forfeiture
vm.prank(userB);
AssetClaims memory attackerClaims = entryPoint.executeRedemption(userB, newNonce,
type(uint256).max);
uint256 attackerReceived = toUint256(attackerClaims.nav);

// userA should have received ~original deposit (yield was forfeited)
assertApproxEqAbs(honestReceived, depositAmount, 2, "honest user receives ~original deposit (yield
forfeited)");

// userB should have received significantly more (kept the yield)
assertGt(attackerReceived, honestReceived, "attacker received more than honest user by bypassing
forfeiture");

// userB kept the full 10% yield that should have been forfeited
// received ≈ depositAmount * 1.10
assertApproxEqAbs(attackerReceived, depositAmount * 110 / 100, 2, "attacker kept ~10% yield that should
have been forfeited");
}

```

ROYCO4-2 | Maple redemptions are checked against RoycoEntryPoint instead of the original owner

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

src/kernels/MaplePoolV2_ST_JT_ExitSharePriceToChainlinkOracle_Kernel.sol#L90

Description:

The Maple kernel models redemption permissioning by validating the share owner whose tranche shares are being burned. In `_preTrancheBalanceUpdate`, a burn (`_to == address(0)`) resolves the user subject to `_from` and queries the Maple permission manager for that address.

```
if (_to == address(0)) {
    users = new address;
    users[0] = _from;
    functionIdToCheck = MAPLE_POOL_TRANSFER_FUNCTION_ID;
}
...
require(

IMaplePoolPermissionManager(IMaplePoolManager(MAPLE_POOL_MANAGER).poolPermissionManager())
    .hasPermission(MAPLE_POOL_MANAGER, users, functionIdToCheck),
    OPERATION_REJECTED_BY_MAPLE_PERMISSION_MANAGER()
);
```

In the queued redemption flow, **RoycoEntryPoint** first pulls the user's tranche shares into itself at request time. At execution time, the entry point then redeems those shares with itself as the owner.

Because the burn is driven from the entry point, `_from` in the Maple kernel resolves to **RoycoEntryPoint** rather than the original requester. The user is only permission-checked once, at request time, during the **transferFrom** into the entry point. After that point, any change to the user's Maple permission status is no longer reflected in the execution of their queued request, and the check effectively collapses to whether **RoycoEntryPoint** itself is permissioned on the Maple pool.

This diverges from the direct redemption path, where the same user would be re-evaluated against the Maple permission manager at the moment of the burn.

Remediation:

The queued redemption flow should preserve the original requester as the compliance subject for Maple permission checks. Execution should apply the same owner-based permission semantics as the direct redemption path, rather than substituting **RoycoEntryPoint** as the redeeming owner.

ROYCO4-4 | SoulBound tranche kernels are not compatible with RoycoEntryPoint async flows

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

Low

Path:

```
src/kernels/Identical_ERC20_ST_JT_ChainlinkToAdminOracle_SoulBoundTrancheShares_Kernel.sol#L20
```

Description:

RoycoEntryPoint assumes it can act as an intermediate operator for tranche shares. In practice, its async flow relies on two behaviors: minting shares to a user from the entry point during deposit execution, and temporarily holding a user's shares during queued redemptions. The SoulBound kernel does not allow either pattern.

```
if (_from == address(0)) {
    require(
        _to == _caller
        || (_to == _getRoycoKernelStorage().protocolFeeRecipient && _caller ==
address(this)),
        TRANCHE_SHARES_ARE_SOUL_BOUND()
    );
} else {
    require(_to == address(0), TRANCHE_SHARES_ARE_SOUL_BOUND());
}
```

If a SoulBound tranche is enabled on the entry point, **executeDeposit()** will revert because the tranche sees **RoycoEntryPoint** as the caller while the shares are minted to a different receiver. **requestRedemption()** will also revert because moving shares from the user into the entry point is a transfer, while the SoulBound kernel only permits burns for non-mint balance updates.

ACRED is one example of a market configured with the SoulBound kernel, so this behavior is relevant if such a tranche is later enabled in the entry point.

Remediation:

If SoulBound markets are intended to be supported, the async deposit and redemption path should be redesigned so it does not depend on disallowed transfer or third-party mint behavior.

ROYCO4-9 | No slippage control on redemption flow

Acknowledged

Severity:

Low

Probability:

Rare

Impact:

High

Path:

src/periphery/RoycoEntryPoint.sol:requestRedemption#L221-L261

Description:

The function **requestRedemption** allows a user to create an asynchronous redemption request that can be filled by any executor. The request saves a **navAtRequestTime** but there is no slippage or sanity check against a user-provided parameter on whether this is an acceptable NAV.

While a large deviation in NAV is unlikely, it could happen that there is some deviation between the moment when the transaction was signed and published versus when it is actually executed, either naturally or forcefully by an attacker.

```
function requestRedemption(
    address _tranche,
    uint256 _shares,
    address _receiver,
    uint64 _executorBonusWAD
)
    external
    override(IRoycoEntryPoint)
    whenNotPaused
    restricted
    returns (uint256 requestNonce, uint32 executableAtTimestamp)
{
    // Validate the redemption request
    require(_shares != 0, ZERO_AMOUNT());
    require(_tranche != address(0) && _receiver != address(0), NULL_ADDRESS());
    require(_executorBonusWAD <= WAD || _executorBonusWAD == type(uint64).max,
INVALID_EXECUTOR_BONUS());

    // Ensure that the tranche is enabled on this entry point
    RoycoEntryPointState storage $ = _getRoycoEntryPointStorage();
    EnrichedTrancheConfig memory config = $.trancheToConfig[_tranche];
    require(config.baseConfig.enabled, TRANCHE_NOT_ENABLED());
```

```

    // Register the user's redemption request with a fresh nonce
    RedemptionRequest storage request = $.userToNonceToRedemptionRequest[msg.sender]
[requestNonce = ++$.lastRequestNonce];
    request.shares = _shares;

    // If the redeeming LP receives the yield accrued, set to MAX_NAV_UNITS so that navAtExecutionTime is
never greater, effectively disabling yield forfeiture
    request.navAtRequestTime =
    (config.baseConfig.yieldRecipient != AccruedYieldRecipient.REDEEMING_LP ?
IRoycoVaultTranche(_tranche).convertToAssets(_shares).nav : MAX_NAV_UNITS);
    request.baseRequest = BaseRequest({
        tranche: _tranche,
        receiver: _receiver,
        executableAtTimestamp: (executableAtTimestamp = uint32(block.timestamp +
config.baseConfig.redemptionDelaySeconds)),
        executorBonusWAD: _executorBonusWAD
    });

    // Transfer the requested amount of tranche shares into the entry point to queue the redemption
    IERC20(_tranche).safeTransferFrom(msg.sender, address(this), _shares);

    // Emit the redemption request event
    emit RedemptionRequested(msg.sender, requestNonce, _tranche, _shares, executableAtTimestamp,
_executorBonusWAD);
}

```

Remediation:

This can be remediated by adding a “minimum NAV” parameter to the **requestRedemption** function that allows the user to specify a minimum NAV value that they are willing to accept at runtime for **navAtRequestTime**.

ROYCO4-6 | Missing burn and burnFrom in deploy scripts

Fixed ✓

Severity:

Informational

Probability:

Very likely

Impact:

Informational

Path:

script/Deploy.s.sol:_buildTrancheRolesConfig#L391-L411

Description:

In the Deploy script, it builds the roles configuration for all contracts, including the tranches in the function `_buildTrancheRolesConfig`.

Each function is assigned a role (since they all have the **restricted** modifier), except for the new **burn** and **burnFrom** functions. These are not assigned.

These functions are to be used by the RoycoEntrypoint contract, but this would currently be impossible if not manually set.

```
function _buildTrancheRolesConfig(address _tranche, uint64 _lpRole) private pure returns
(IRoycoFactory.RolesTargetConfiguration memory) {
    bytes4[] memory selectors = new bytes4[](7);
    uint64[] memory roleValues = new uint64[](7);

    selectors[0] = IRoycoVaultTranche.deposit.selector;
    roleValues[0] = _lpRole;
    selectors[1] = IRoycoVaultTranche.redeem.selector;
    roleValues[1] = _lpRole;
    selectors[2] = IRoycoAuth.pause.selector;
    roleValues[2] = ADMIN_PAUSER_ROLE;
    selectors[3] = IRoycoAuth.unpause.selector;
    roleValues[3] = ADMIN_PAUSER_ROLE;
    selectors[4] = UUPSUpgradeable.upgradeToAndCall.selector;
    roleValues[4] = ADMIN_UPGRADER_ROLE;
    selectors[5] = IRoycoVaultTranche.seizeShares.selector;
    roleValues[5] = TRANSFER_AGENT_ROLE;
    selectors[6] = IRoycoVaultTranche.seizeAndRedeemShares.selector;
    roleValues[6] = TRANSFER_AGENT_ROLE;

    return IRoycoFactory.RolesTargetConfiguration({ target: _tranche, selectors: selectors, roles: roleValues });
}
```

Remediation:

Add the **burn** and **burnFrom** functions to the **_buildTrancheRolesConfig** function with the correct roles configured, such as a specific Entrypoint role.

ROYCO4-11 | Gas: Skip blacklist checks on transfers if not enabled

Fixed 

Severity:

Informational

Probability:

Unlikely

Impact:

Informational

Path:

src/kernels/base/RoycoKernel.sol:preTrancheBalanceUpdateHook#L539-L568

Description:

The RoycoKernel will perform blacklist checks upon every transfer of tranche shares using `preTrancheBalanceUpdateHook`. However, the blacklist is not always enabled.

The function `isBlacklisted` first checks if it enabled using `$.isBlacklistEnabled` and only then does it check the `isBlacklisted` mapping.

Nonetheless, the function `isBlacklisted` is still called 3 times in `preTrancheBalanceUpdateHook` for the caller, sender and receiver. So if the blacklist is not even enabled, it would still do 3 separate SLOADs to check if the blacklist is enabled or not.

```
function preTrancheBalanceUpdateHook(address _caller, address _from, address _to, uint256 _value)
    external
    override(IRoycoKernel)
    onlyTranche
    whenNotPaused
{
    // Check if caller is blacklisted or not
    require(!isBlacklisted(_caller), ACCOUNT_BLACKLISTED(_caller));

    // Check if the sender is blacklisted if not a mint
    require(_from == address(0) || !isBlacklisted(_from), ACCOUNT_BLACKLISTED(_from));

    // Check if the recipient is blacklisted if not a redeem
    if (_to != address(0)) {
        require(!isBlacklisted(_to), ACCOUNT_BLACKLISTED(_to));

        // If transferring shares, ensure that the recipient is a whitelisted LP for the tranche
        // It is assumed that the sender is already a whitelisted LP
        if (ENFORCE_TRANCHE_SHARES_TRANSFER_WHITELIST) {
            address authority = authority();
            // Check if the to address can call the deposit function on the tranche
        }
    }
}
```

```

    // @dev msg.sender is the tranche address
    (bool isWhitelistedTrancheLP,) = IAccessManager(authority).canCall(_to, msg.sender,
IRoycoVaultTranche.deposit.selector);
    require(_to != authority && isWhitelistedTrancheLP,
ACCOUNT_NOT_WHITELISTED_TRANCHE_LP(_to));
  }
}

// Call the market specific pre-balance update hook
_preTrancheBalanceUpdate(_caller, _from, _to, _value);
}

```

Remediation:

We recommend to lift the **\$.isBlacklistEnabled** check into the **preTrancheBalanceUpdateHook** function so you can skip the blacklist checks in one time.

This would save gas on each transfer of tranche shares.

ROYCO4-8 | Gas: Pre sync accounting should use cached variables

Acknowledged

Severity:

Informational

Probability:

Very likely

Impact:

Informational

Path:

src/accountant/RoycoAccountant.sol:_previewSyncTrancheAccounting#L403-L620

Description:

In the function `_previewSyncTrancheAccounting`, both `betaWAD` and `coverageWAD` are used. They are loaded into local cache variables on lines 559 and 560.

However, these variables are also used prior to this point in a calculation on line 527.

The caching should be moved upwards so the cached versions can be used in the calculation on line 527.

```
IYDM($.ydm).previewJTYieldShare(initialMarketState, $.lastSTRawNAV, $.lastJTRawNAV, $.betaWAD,
$.coverageWAD, $.lastJTEffectiveNAV);
[.]
uint96 betaWAD = $.betaWAD;
uint64 coverageWAD = $.coverageWAD;
```

Remediation:

See description.

Severity:

Informational

Probability:

Very likely

Impact:

Informational

Path:

src/periphery/RoycoEntryPoint.sol:executeDeposits#L110-L126

src/periphery/RoycoEntryPoint.sol:cancelDepositRequests#L188-L194

src/periphery/RoycoEntryPoint.sol:executeRedemptions#L264-L280

Description:

All batch function variants in RoycoEntryPoint will call the public single function directly. But the single function all have the **whenNotPaused** and **restricted** modifier, which means that it will check these modifiers on each execution line of the batch.

```
function executeDeposits(
    address _user,
    uint256[] calldata _requestNonces,
    TRANCHE_UNIT[] calldata _assetsToDeposit
)
    external
    override(IRoycoEntryPoint)
    returns (uint256[] memory trancheSharesMinted)
{
    // Execute the user specified deposit requests
    uint256 numRequestsToExecute = _requestNonces.length;
    require(numRequestsToExecute == _assetsToDeposit.length, ARRAY_LENGTH_MISMATCH());
    trancheSharesMinted = new uint256[](numRequestsToExecute);
    for (uint256 i = 0; i < numRequestsToExecute; ++i) {
        trancheSharesMinted[i] = executeDeposit(_user, _requestNonces[i], _assetsToDeposit[i]);
    }
}
```

Remediation:

We recommend to consider splitting this single function into one internal function with all current logic except for the modifier and have a public single function and a public batch function with the modifiers that call the internal function directly.

ROYCO4-12 | Gas: Remove redundant calculations

Fixed ✓

Severity:

Informational

Probability:

Very likely

Impact:

Informational

Path:

src/kernels/base/RoycoKernel.sol:jtMaxWithdrawable#L257-L293

Description:

The function **jtMaxWithdrawable** calculates the max withdrawable amount for a JT holder.

In the function it uses **stConvertTrancheUnitsToNAVUnits(jtNotionalClaims.stAssets)** and **jtConvertTrancheUnitsToNAVUnits(jtNotionalClaims.jtAssets)** to convert notional claim assets back to NAV for both ST and JT claims respectively.

However, it does this twice, both on lines 283-284 and on lines 287-288.

```
function jtMaxWithdrawable(address _owner)
    public
    view
    virtual
    override(IRoycoKernel)
    returns (
        NAV_UNIT claimOnStNAV,
        NAV_UNIT claimOnJtNAV,
        NAV_UNIT stMaxWithdrawableNAV,
        NAV_UNIT jtMaxWithdrawableNAV,
        uint256 totalTrancheSharesAfterMintingFees
    )
{
    // If the owner is blacklisted, return zero claims
    if (isBlacklisted(_owner)) return (ZERO_NAV_UNITS, ZERO_NAV_UNITS, ZERO_NAV_UNITS,
    ZERO_NAV_UNITS, 0);

    // Get the total claims the junior tranche has on each tranche's assets
    SyncedAccountingState memory state;
    AssetClaims memory jtNotionalClaims;
    (state, jtNotionalClaims, totalTrancheSharesAfterMintingFees) =
    previewSyncTrancheAccounting(TrancheType.JUNIOR);

    // Get the max withdrawable ST and JT assets in NAV units from the accountant consider coverage
    requirement
```

```

(, NAV_UNIT stClaimableGivenCoverage, NAV_UNIT jtClaimableGivenCoverage) =
IRoycoAccountant(ACCOUNTANT)
    .maxJTWithdrawalGivenCoverage(
        state.stRawNAV,
        state.jtRawNAV,
        stConvertTrancheUnitsToNAVUnits(jtNotionalClaims.stAssets),
        jtConvertTrancheUnitsToNAVUnits(jtNotionalClaims.jtAssets)
    );

claimOnStNAV = stConvertTrancheUnitsToNAVUnits(jtNotionalClaims.stAssets);
claimOnJtNAV = jtConvertTrancheUnitsToNAVUnits(jtNotionalClaims.jtAssets);

// Bound the claims by the max withdrawable assets globally for each tranche and compute the
cumulative NAV
stMaxWithdrawableNAV = stClaimableGivenCoverage;
jtMaxWithdrawableNAV = jtClaimableGivenCoverage;
}

```

Remediation:

We recommend moving the calculation up and caching the result so that it can be used in both places.

hexens x ROYCO

